

第七次作业反馈

林洛昊

2024.11

1 相关知识回顾

终于来到树这一章了，树还是一个相当重要的数据结构的，大家后面在学堆排序的时候，也会用到树这样一个结构，包括大三上其它算法课的时候，也会接触到 AVL 树，红黑树，B 树，B+ 树等五花八门的东西，我们目前学的二叉树是以上只是的基础，我在此列举一下 ppt 上的相关内容

首先是树的一些基本性质：

• 树的若干性质

- 1 结点数为 n 的树，每个结点的度为 d_i ，则 $n = \sum_{i=1}^n d_i + 1$
- 2 度为 k 的树，第 i 层至多有 k^{i-1} 个结点
 - 证明：数学归纳法
- 3 度为 k 且深度为 h 的树，总结点数不超过 $\frac{k^h-1}{k-1}$
- 4 度为 k 且结点数为 n 的树，深度不小于 $\lceil \log_k(n(k-1)+1) \rceil$ ，其中 $\lceil \cdot \rceil$ 表示取上整 (ceiling)

• 二叉树的若干性质

- 1 二叉树第 i 层上至多有 2^{i-1} 个结点
- 2 深度为 h 的二叉树至多有 2^h-1 个结点
- 3 若二叉树的叶子结点数为 n_0 ，度为2的结点数为 n_2 ，则 $n_0=n_2+1$

这些性质主要是考试的时候可能会有选填考这些，当然，你们上图论的话可能也会用到。

接下来再看看树结构的两种定义：

```

typedef int TElemType;
const unsigned int MAX_TREE_SIZE = 100;
typedef struct {
    TElemType nodes[MAX_TREE_SIZE];
    int n;
} SqBiTree;

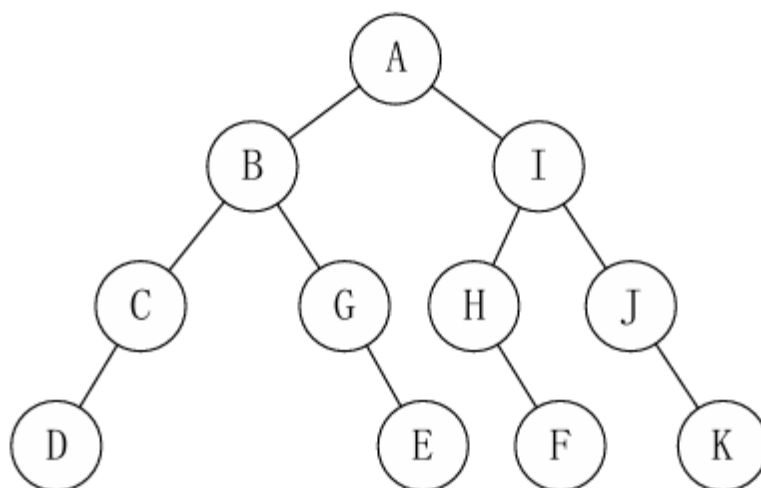
typedef struct BiTNode {
    TElemType data;
    BiTNode *lchild, *rchild;
} *BiTree;

```

这两种定义方式各有利弊，大家灵活使用，怎么方便怎么来。对于遍历来说，利用递归的思想可以比较方便地以各种顺序（不包含层序）来遍历链式存储的二叉树。

2 作业题目解答

5.10 已知某二叉树的中序序列为 DCBGEAHFIJK，后序序列为 DCEGBFHKJIA。请画出该二叉树



5.14 试写出复制一棵二叉树的算法

这题非常简单，我们用递归的思想先序遍历一遍原来的二叉树，顺便创建新的树和它的子节点即可。注意要先考虑空指针的情况。

```

void CopyBiTree(BiTree T, BiTree& NewT)
{
    if (T == NULL)
    {
        NewT = NULL;
        return;
    }
    else

```

```

    {
        NewT = new BiTNode;
        NewT->data = T->data;
        CopyBiTree(T->lchild, NewT->lchild);
        CopyBiTree(T->rchild, NewT->rchild);
    }
}

```

5.18 写出判断两棵给定二叉树是否相似的算法。A 和 B 相似，是指 AB 皆空，或者 AB 皆不空且 A 和 B 的左右子树分别对应相似

这题同样还是用递归的思想解决，跟 5.14 其实很像。

```

bool SimilarBiTree(BiTree T1, BiTree T2)
{
    if (T1 == NULL && T2 == NULL) // 左右皆空
    {
        return true;
    }
    else
    {
        if (T1 == NULL || T2 == NULL)
        {
            return false;
        }
        else
        {
            return SimilarBiTree(T1->lchild, T2->lchild) && SimilarBiTree(T1->rchild,
                T2->rchild);
        }
    }
}

```

5.26 求先序遍历序列的第 k 个元素算法

定义一个外部的数组来存储元素，然后递归先序遍历元素

```

void PreOrder(BiTree T, vector<TElemType>& res)
{
    if (T == NULL)
    {
        return;
    }
    else
    {
        res.push_back(T->data);
        PreOrder(T->lchild, res);
        PreOrder(T->rchild, res);
    }
}

TElemType PreOrderVal(BiTree T, int k)
{
    vector<TElemType> res;

```

```
    PreOrder(T, res);  
    return res[k - 1];  
}
```

3 补充阅读资料

Hello 算法-第七章树